

EV Battery Monitoring System

By: Dhanush Raja A K

This is the battery monitoring logic I built for our EV project. It runs on a 4S Li-ion pack (15V nominal, 10Ah) and keeps an eye on voltage, current, temperature and smoke level, then works out things like charge percentage, load, and how much runtime is left. All of it gets shown on the LCD, and it flags warnings if something looks off.

Putting this doc together mainly so the team doesn't have to keep asking me what each threshold means or dig through the code to figure out the formulas.

Contents

1. [Battery specs we're using](#)
 2. [What the system reads](#)
 3. [Power calculation](#)
 4. [SOC \(charge %\) estimation](#)
 5. [Battery health status](#)
 6. [Charging vs discharging](#)
 7. [Load level](#)
 8. [Temperature status](#)
 9. [Smoke detection](#)
 10. [Runtime left](#)
 11. [Overall status](#)
 12. [All thresholds in one place](#)
 13. [What shows on the LCD](#)
 14. [Things to keep in mind](#)
-

Battery specs we're using

Parameter	Value
Configuration	4S Li-ion
Full charge voltage	16.8 V
Nominal voltage	14.8 V
Minimum safe voltage	12.0 V
Capacity	10 Ah

Standard numbers for a 4-cell Li-ion pack. If we end up swapping the pack for something else, these are the values to update first since everything else depends on them.

What the system reads

Four sensor inputs go into all the logic below:

- **Voltage** off the pack, straight from a voltage sensor
 - **Current**, signed (positive when discharging, negative when charging, but check your sensor's wiring since this can come out reversed)
 - **Temperature** near the pack
 - **Smoke/gas level** as a raw ADC reading from the MQ135
-

1. Power calculation

Nothing fancy here, just:

```
power = voltage * current;
```

Some example numbers to sanity-check against:

Voltage	Current	Power
15.2 V	2 A	30.4 W
14.5 V	5 A	72.5 W
13.8 V	8 A	110.4 W

LCD shows:

```
Power : 72.5 W
```

2. SOC (charge %) estimation

We're doing this off open-circuit voltage since we don't have a coulomb counter set up. Worth knowing that this means the SOC reading will dip a bit under heavy load even if the battery still has charge left, that's just voltage sag and not an actual capacity drop.

Voltage Range	Estimated SOC
>= 16.8 V	100%
16.4 - 16.8 V	90%
16.0 - 16.4 V	80%
15.6 - 16.0 V	70%
15.2 - 15.6 V	60%
14.8 - 15.2 V	50%
14.4 - 14.8 V	40%
14.0 - 14.4 V	30%
13.6 - 14.0 V	20%
12.0 - 13.6 V	10%
< 12.0 V	0%

LCD shows:

```
SOC : 70%
```

3. Battery health status

Just bucketing the SOC number into something more readable:

SOC Range	Status
> 70%	GOOD
30 - 70%	LOW
< 30%	CRITICAL

LCD shows:

```
Battery : GOOD
```

4. Charging vs discharging

Based on current sign:

```
if (current > 0.2)
    state = DISCHARGING;
else if (current < -0.2)
    state = CHARGING;
else
    state = IDLE;
```

(Used 0.2A as a dead zone so it doesn't flicker between states when current is near zero. If the sensor's polarity is flipped on your board, just swap the two comparisons.)

LCD shows:

```
State : Discharging
```

5. Load level

How hard the battery's being worked, based on discharge current:

Current Range	Load Level
0 - 2 A	LOW
2 - 5 A	MEDIUM
5 - 8 A	HIGH
> 8 A	VERY HIGH

LCD shows:

```
Load : HIGH
```

6. Temperature status

Temperature Range	Status
< 35 C	NORMAL
35 - 45 C	WARM
45 - 55 C	HOT
> 55 C	CRITICAL

LCD shows:

```
Temp : WARM
```

7. Smoke detection

This one reads a raw ADC value off the MQ135. Heads up: these numbers below are just starting points, not calibrated yet. We should run the sensor in clean air first and note down its baseline reading before trusting these thresholds for anything real.

ADC Value	Status
< 150	SAFE
150 - 250	WARNING

> 250

CRITICAL

LCD shows:

```
Smoke : SAFE
```

TODO: calibrate against our actual sensor unit, current numbers are rough guesses.

8. Runtime left

Simple version (assumes battery starts full):

```
remaining_time = capacity / current;
```

Current	Remaining Time
1 A	10 hours
2 A	5 hours
5 A	2 hours
10 A	1 hour

Better version, factors in whatever SOC we're actually at:

```
remaining_time = (SOC * capacity) / current;
```

Quick example: if SOC is 60% and we're pulling 2A off a 10Ah pack, remaining capacity is 6Ah, so $\text{remaining_time} = 6 / 2 = \mathbf{3 \text{ hours}}$.

LCD shows:

```
Time Left : 2 hr
```

9. Overall status

This is what pulls everything together into one line for the user:

Condition	Status
All parameters within normal range	SAFE
Temperature warm OR voltage low	WARNING
Smoke detected OR temperature critical	EMERGENCY

LCD shows:

```
STATUS : SAFE
```

Quick summary of what feeds into what

Input	Derived Output
Voltage	Estimated SOC
Voltage + Current	Power
Current	Load level
Current sign	Charging / discharging

Temperature

Temperature status

Smoke ADC	Smoke status
Capacity + Current + SOC	Remaining runtime
All of the above	Overall system status

All thresholds in one place

Handy reference if you don't want to scroll through every section above:

Parameter	Normal	Warning	Critical
Voltage	14.0 - 16.8 V	12.5 - 14.0 V	< 12.5 V or > 16.8 V
Current	0 - 5 A	5 - 8 A	> 8 A
Temperature	< 35 C	35 - 45 C	> 55 C
Smoke (ADC)	< 150	150 - 250	> 250
SOC	> 70%	30 - 70%	< 30%

What shows on the LCD

Screen cycles through these on a timer:

1. Temp: 32.5C Volt: 15.1V
2. Curr: 2.8A Power: 42.3W
3. SOC: 70% Load: MED
4. State: DISCHARGING
5. Status: SAFE

Things to keep in mind

- SOC is purely voltage-based right now. It'll drift a bit under load. If we have time later, adding a coulomb counter would make this way more accurate.
- Smoke thresholds are placeholders, still need to calibrate the MQ135 for real.
- Double check current sensor polarity on your specific board before trusting the charge/discharge state.
- All the thresholds here are easy to tweak once we've done actual testing, don't treat these as final.

If anyone wants to change a threshold or the logic, just open a PR and mention why (test results, datasheet reference, whatever).